Zaryah Plus — System Architecture

How the platform is built — and where a new feature fits in. For first-time developers.

Audience	Developers new to this codebase. Assumes you can read code but are new to this stack.
Scope	The whole platform: frontend, backends, database, security, languages, and how they connect.
Sakinah	Appears only in Part 9 as an example of how a **new feature** plugs into all of this.

> **How to read this**
>
> Parts 1–2: the big picture and the toolset. Parts 3–4: the frontend and how it is organised into features. Parts 5–7: data, backends, and security. Part 8: how a request actually travels. Part 9: where a new feature (Sakinah) fits. Part 11: a dictionary — flip to it whenever a word is unfamiliar.

1. The Big Picture

Zaryah Plus is a Muslim “super-app” — many features (prayer times, Quran, an AI companion called Raya, finance tools, gratitude, social, and more) inside one product. It is built from a small number of moving parts that the rest of this guide unpacks.

Three ideas explain almost everything:

- **One frontend, two surfaces.** A single frontend codebase written in React. The website and the mobile apps (iOS + Android) are all built from that **same** code — a tool called Capacitor wraps it into phone apps.
- **The frontend talks to backends and a database.** The browser is never trusted with secrets or important decisions, so heavy/sensitive work happens on servers, and data is stored in a cloud database (Firestore).
- **The app is a collection of “features.”** Each feature (auth, quran, connections, …) is a self-contained folder. Adding capability to the app usually means adding another feature folder.

Read the diagram top-down: users open the frontend; the frontend reads/writes the Firebase database directly for simple things, and calls backend servers for trusted things. Everything below the frontend is shared platform.

2. The Languages & Tools

Tool / language	What it is, in plain words	Layer
TypeScript	JavaScript with type labels so the editor catches mistakes early. The frontend language.	Frontend
React	Turns code into screens, out of reusable pieces called components.	Frontend
Vite	The dev server + builder — runs the app locally and packages it for release.	Frontend
Tailwind CSS	Styling by adding small class names instead of writing separate CSS files.	Frontend
Zustand	Holds small app-wide state in memory (e.g. who is logged in).	Frontend
TanStack Query	Fetches server data and caches it so screens load fast and stay fresh.	Frontend
i18next	Translations — the app speaks several languages.	Frontend
Capacitor	Wraps the React app into real iOS + Android apps.	Frontend
Firebase	Google's backend-in-a-box: login (Auth), database (Firestore), file storage, small server functions.	Platform
Firestore	The cloud database. Stores data as documents grouped into collections (like rows in tables).	Data
Python + FastAPI	The language + framework used to build the backend servers here.	Backend
Pinecone / Redis	A search index for AI features / a fast cache. Used by the existing AI backend.	Backend

The one-line version

Frontend = TypeScript + React (built by Vite). Backends = Python + FastAPI and **Firebase Cloud Functions. Login + database + storage = Firebase / Firestore.**

3. The Frontend

Everything frontend lives under *frontend/src/*. The folders that matter:

Folder (inside frontend/src/)	What lives here
features/	**The heart of the app.** One folder per feature (auth, quran, connections, …).
app/	App-wide wiring: *router.tsx* (which URL shows which screen) and *providers.tsx* (global context).
core/	Shared building blocks: *stores/* (Zustand state, e.g. the auth store), shared services, theme.
config/	*firebase.config.ts* — where the app connects to Firebase (Auth, Firestore, Storage).
lib/	Shared helpers. Most important: *api.ts* — how the frontend calls a backend (Part 5).
components/ , hooks/	Generic UI pieces and reusable React logic shared across features.
i18n/	Translation strings.

The app boots from *main.tsx* → wraps everything in *app/providers.tsx* (data caching + auth initialisation) → *app/router.tsx* decides which feature's screen to show for the current URL. Authenticated screens render inside a shared layout (sidebar + nav) and are protected by an **AuthGuard** so only logged-in users get in.

4. The Feature System

The app is composed of **feature modules**. Around twenty are live today, for example:

Core	Identity & social	Islamic & AI	Finance & more
auth	kyc	chatbot (Raya)	screener
home	public-profile	quran	wallet
navigation	connections	prayer-times	barakah-labs
profile / settings	dms	zakat	admin

Every feature folder follows the same internal shape, which makes any feature easy to navigate once you know one:

```
features/<name>/
  pages/        # full screens (one per route)
  components/   # smaller UI pieces used by those pages
  services/     # the logic: talking to Firestore or to a backend
  hooks/        # reusable React logic
  types/        # the TypeScript shapes of the data
  stores/       # (optional) Zustand state local to the feature
```

A page renders UI and calls a **hook**; the hook calls a **service**; the service is the only place that actually talks to data. A feature is connected to the app by (1) adding routes in *app/router.tsx* and (2) adding menu entries in *features/navigation/*. **Adding a new capability to Zaryah usually means adding one more feature folder this way.**

5. How The App Reaches Data & Backends

There are exactly two ways the frontend gets or changes data:

Way 1 — straight to Firestore (the database)

A service reads/writes Firestore directly from the browser using the Firebase SDK (set up in *config/firebase.config.ts*). The **security rules** on Firestore decide whether each read or write is allowed (Part 7). This suits simple, non-sensitive data.

Way 2 — through a backend, via lib/api.ts

For trusted or heavy work, the frontend calls a backend server. The shared helper *lib/api.ts* does this and **attaches the logged-in user's Firebase ID token to every request**, so the backend knows who is calling. The backend address comes from the *VITE_BACKEND_URL* setting in *.env.local*.

6. The Backends

Zaryah deliberately runs more than one backend, each suited to its job:

Backend	Language	What it is for
AI service	Python / FastAPI	The AI companion (Raya), search over Islamic books, finance tools. Deployed on a cloud host; uses a vector index (Pinecone) and a cache (Redis).
Cloud Functions	Node.js	Tiny serverless functions that react to database events (e.g. send a notification when a record is written), living close to Firestore.

Backends authenticate callers by verifying the Firebase ID token, and they write sensitive data using admin access (which bypasses client rules — that is exactly why such work belongs on a server, not in the browser). New backend services can be added alongside these when a feature needs its own

trusted server.

7. Security: Where The Rules Live

The browser is never trusted. The server and the rules are.

Anything a user must not be able to do has to be blocked on the **server** or by **Firestore security rules** — never just hidden in the UI, because anyone can inspect and bypass browser code.

- **Firebase Auth** establishes *who* a user is and issues an ID token. That token is what both Firestore rules and the backends check.
- **Firestore security rules** (*firestore.rules*) decide, per collection, who may read or write. Sensitive collections are typically locked so only a backend (admin access) may write them.
- **Backends** verify the token, check the caller is allowed, and are the only writers of sensitive data. This “the server decides” posture is called **server-authoritative**.

8. Two Request Journeys

Journey A — reading simple data straight from Firestore

```
Browser (a service in features/...)
   |   reads/writes via the Firebase SDK
   v
Firestore  --->  security rules check: allowed? yes/no
   |
   v
data returned to the screen
```

Journey B — a trusted action through a backend

```
Browser  --calls lib/api.ts-->  (attaches the Firebase ID token)
   |
   v
Backend server (FastAPI)
   1. verify the token  ->  who is this user?
   2. check they're allowed to do this
   3. read/write Firestore with admin access
   4. return the result
   |
   v
Browser updates the screen
```

9. Where A New Feature (Sakinah) Fits

Sakinah is simply **another feature** added to everything above — it does not change the architecture. Concretely, it slots in at three points already shown in this guide:

- **Frontend:** a new *features/sakinah/* module (Part 4 pattern), wired into *app/router.tsx* and the navigation, reusing the existing auth, Firebase config, and design system.
- **Backend:** because it handles verification, matching, and safety, it gets its **own new server-authoritative FastAPI backend** — added alongside the existing servers (Part 6), reached from the frontend through *lib/api.ts* (Part 5, Journey B).
- **Data & security:** its own Firestore collections, with sensitive ones locked to backend-only writes (Part 7).

```
  FRONTEND feature modules:  auth . quran . connections ... [ sakinah ]   <- new
                                                              |
                                          lib/api.ts (token)  |
                                                              v
  BACKENDS:  FastAPI AI service . Cloud Functions . [ Sakinah FastAPI ]   <- new
                                                              |
                                                admin writes  v
  DATA:      Firestore  (existing collections) . [ Sakinah collections ]  <- new
```

> **The takeaway**
> Nothing about the platform changes for Sakinah. It is one more feature folder on the frontend and one more server-authoritative backend behind it — following the exact same patterns every other feature uses.

10. Start-Here File Map

To orient yourself in the frontend, open these in order:

File	Why open it
frontend/src/main.tsx	The entry point — where the app starts.
frontend/src/app/providers.tsx	The global wrappers (data caching, auth init).
frontend/src/app/router.tsx	The map of URL → screen.
frontend/src/config/firebase.config.ts	How the app connects to Firebase (Auth + Firestore).
frontend/src/lib/api.ts	How the frontend calls a backend (token attached).
frontend/src/core/stores/	Zustand stores — e.g. the auth store that knows the current user.
frontend/src/features//	Any existing feature folder — read one as an example of the pattern.

11. Dictionary

Word	Plain meaning
Frontend	The part that runs in the browser / app — what the user sees.
Backend	A server program that does trusted or heavy work for the app.
Database	Where data is stored permanently. Here: Firestore.
React	The toolkit that builds the frontend out of reusable components.
Component	A reusable piece of a screen, written as a function.
TypeScript (.ts/.tsx)	JavaScript with type labels; .tsx files also hold screen layout.
Vite	Runs the frontend locally and builds it for release.
Capacitor	Wraps the React app into iOS + Android apps.
Zustand	Small in-memory app-wide state (e.g. current user).
TanStack Query	Fetches + caches server data for the frontend.
Feature module	A self-contained folder under features/ implementing one capability.
Firebase	Google’s backend-in-a-box: Auth + Firestore + Storage + Functions.
Firestore	The cloud database: documents grouped into collections.
Collection / Document	A group of records / a single record.
Auth / ID token	The login system / a signed string proving who the user is.
Security rules	Firestore’s rulebook for who may read/write what.
Cloud Function	A tiny serverless function that reacts to events (Node.js).
FastAPI	The Python framework used to build backend servers here.
Pinecone / Redis	An AI search index / a fast cache used by the AI backend.
Server-authoritative	The server, not the browser, makes and enforces decisions.
lib/api.ts	The frontend helper that calls a backend with the user’s token attached.
VITE_BACKEND_URL	The setting telling the frontend where a backend lives.

If you remember one thing

Zaryah is **one React frontend made of feature folders**, talking to **Firebase** (login + database) and **FastAPI backends** (trusted work), where **the server — not the browser — makes every real decision.** A new feature is one more folder and, when it needs trust, one more backend.